

When Not to Use

In some cases, encapsulation is ill-advised due to performance considerations. These cases are rare but when they happen, this circumstance is very important

Say that you have a graphical editor that contains objects possessing x- and y-coordinates. These fields are unlikely to change in the future. What's more, the program involves a great many different objects in which these fields are present. So accessing the coordinate fields directly saves significant CPU cycles that would otherwise be taken up by calling access methods

As an example of this unusual case, there's the Point class in Java. All fields of this class are public

How to Refactor

.Create a getter and setter for the field

Find all invocations of the field. Replace receipt of the field value with the getter, and replace setting of new field values with the setter

.After all field invocations have been replaced, make the field private

Next Steps

Encapsulate Field is only the first step in bringing data and the behaviors involving this data closer together. After you create simple methods for access fields, you should recheck the places where these methods are called. It's quite possible that the code in these areas would look more appropriate in the access methods

Encapsulate Field

مشکل (Problem)

یه فیلد public داره.

راه حل (Solution)

فیلد رو **private** کن و متدهای دسترسی (getter/setter) براش بساز.

مثال از صفحه

قبل:

csharp

```
class Person
{
    public string name
{
```

بعد:

csharp

```
class Person
{
    private string name

    public string Name
    {
        get { return name; }
        set { name = value; }
    }
}
```

چرا ریفکتور کنیم؟ (Why Refactor)

یکی از ستون‌های شی‌گرایی، کپسوله‌سازی هست — توانایی پنهان کردن داده‌های شیء. بدونش، همه اشیاء می‌تونن داده‌های شیء تو رو بدون هیچ بررسی و کنترلی بگیرن و تغییر بدن! داده از رفتار جدا میشه، ماژولاریتی خراب میشه و نگهداری پیچیده میشه.

مزایا (Benefits)

مزیت	توضیح
داده و رفتار کنار هم	نگهداری و توسعه آسون‌تر
عملیات پیچیده	می‌تونی منطق (اعتبارسنجی، لاگ، ...) به getter/setter اضافه کنی

When Not to Use

در موارد نادر، کپسوله‌سازی به خاطر عملکرد (Performance) توصیه نمیشه. مثلاً به ویرایشگر گرافیکی با کلی شیء که مختصات x و y دارن. این فیلدها تغییر نمی‌کنن و صدا زدن getter/setter برای میلیون‌ها بار CPU رو هدر میده. در این صورت public منطقه. (مثال: کلاس Point در Java)

How to Refactor (چطور انجام بدیم)

قدم	توضیح
۱	برای فیلد getter و setter بساز
۲	تمام استفاده‌های فیلد رو با getter/setter جایگزین کن
۳	بعد از جایگزینی همه، فیلد رو private کن

Next Steps

Encapsulate Field فقط قدم اوله. بعدش باید جاهایی که از این متدها استفاده میشه رو بررسی کنی — شاید اون منطق هم باید به خود کلاس منتقل بشه.

تمرین

این کد رو با **Encapsulate Field** اصلاح کن:

csharp

```
public class Product
{
    ;public string Name
    ;public decimal Price
    ;public int Quantity

    // استفاده فعلی:
    ;()var p = new Product
    ;"p.Name = "Laptop
    ;p.Price = -5000 // ❌ قیمت منفی!
    ;p.Quantity = -10 // ❌ تعداد منفی!
```

راهنمایی:

- فیلدها رو private کن
- Property با getter/setter بساز
- توی setter Price و Quantity اعتبارسنجی کن (منفی نباشن)

بفرست تا بررسی کنیم! 🙌

خودت حلش کن

کد اصلاح شده با Encapsulate Field

csharp

```
public class Product
{
    ;private string name
    ;private decimal price
    ;private int quantity

    public string Name
    {
        get { return name; }
    }
}
```

```
        set { name = value; }
    }

    public decimal Price
    {
        get { return price; }
        set
        {
            if (value < 0)
                throw new ArgumentException("قیمت نمی‌تواند منفی باشد");
            price = value;
        }
    }

    public int Quantity
    {
        get { return quantity; }
        set
        {
            if (value < 0)
                throw new ArgumentException("تعداد نمی‌تواند منفی باشد");
            quantity = value;
        }
    }
}
```

مقایسه

نسخه اصلاح	کد اصلی	
✓ private	public	فیلدها
✓ غیرممکن	✗ ممکن بود	قیمت منفی
✓ غیرممکن	✗ ممکن بود	تعداد منفی
✓ فقط setter رو تغییر بده	باید همه جا رو عوض کنی	تغییر بعدی